HANDS-ON LAB SERIES

Berlin · Oct 2026

The Agentic Data Cloud

Governance and real-time data as the foundation agents are built on — not bolted on after.

LAB 1

Foundation & Governance

BigQuery, Knowledge Catalog, and policy-tag masking — governance built into the data platform itself.

- ✓ Business glossary grounding, not hardcoded rules
- ✓ Column masking enforced by identity, not by prompt
- ✓ BigQuery Graph for multi-hop analysis

LAB 2

Zero-ETL & Orchestration

AlloyDB, Spanner, and a multi-agent router — federated reads and structural write boundaries across three engines.

- ✓ Live federation into BigQuery, no pipelines
- ✓ Read-only and write-capable agents, structurally separated
- ✓ Spanner Graph vs. BigQuery Graph — right engine per question

Google Cloud

WHY WE ARE HERE

Every company wants AI agents on its data. Three things go wrong.

Confident wrong answers

THE BUSINESS PROBLEM

Revenue is reported 4× too high — and nobody notices.

THE TECHNICAL CAUSE

Tables hold columns, not business rules. The agent guesses what “revenue” means.

WHAT WE DO TODAY

The rule lives in a catalog the agent must read first.

Lab 1

Agents see or do too much

THE BUSINESS PROBLEM

Customer emails end up in a spreadsheet. A helpful agent deletes data.

THE TECHNICAL CAUSE

Safety written in a prompt can be talked around.

WHAT WE DO TODAY

The platform enforces it: masking and permissions per identity.

Lab 1 + 2

Data in many systems

THE BUSINESS PROBLEM

Orders in one database, finance in another. The reports disagree.

THE TECHNICAL CAUSE

Copy pipelines are slow, costly and always a little out of date.

WHAT WE DO TODAY

BigQuery reads the other databases live — no copies.

Lab 2

Why now, in every industry and every country: agents are moving from answering to acting. A wrong answer used to be one bad report. Now it can be an action — repeated a thousand times.

THE BIG IDEA

Before an AI agent answers a question, it needs two things:

1. The right rules

What does **“revenue”** actually mean here?

2. The right permissions

What is **this specific person** allowed to see?

TAKEAWAY

Today you'll build that foundation first **(Lab 1)**, then connect more data sources to it **(Lab 2)** — and see that the rules and permissions **never have to be rebuilt**.

TODAY

Agenda



13:00	15 min	**Welcome, sign in**	Why we are here · the two labs · sign in to Google Cloud and agy · the tools in 2 minutes
13:15	20 min	**Lab 1 briefing**	BigQuery, Knowledge Catalog, masking, graph — and how to work in the lab
13:35	60 min	**Lab 1 — your turn**	Build a governed agent, then try to break it
14:35	15 min	**Lab 2 briefing**	AlloyDB, Spanner, live federation, the router
14:50	65 min	**Lab 2 — your turn**	Three databases, one question, one agent that may write
15:55	5 min	**Close**	What you built, and what to take home

☕ **Take a break whenever you want — water, coffee, air.**

✋ **Stuck for more than 5 minutes? Ask a facilitator.**

Google Cloud

TWO LABS, ONE PLATFORM

What You'll Build Today

Lab 1 — Foundation

BigQuery, Knowledge Catalog, and policy-tag masking — governance built into the data platform itself.

- ✓ Business glossary grounding, not hardcoded rules
- ✓ Column masking enforced by identity, not by prompt
- ✓ BigQuery Graph for multi-hop analysis

Lab 2 — Zero-ETL & Orchestration

AlloyDB, Spanner, and a multi-agent router — federated reads and structural write boundaries across three engines.

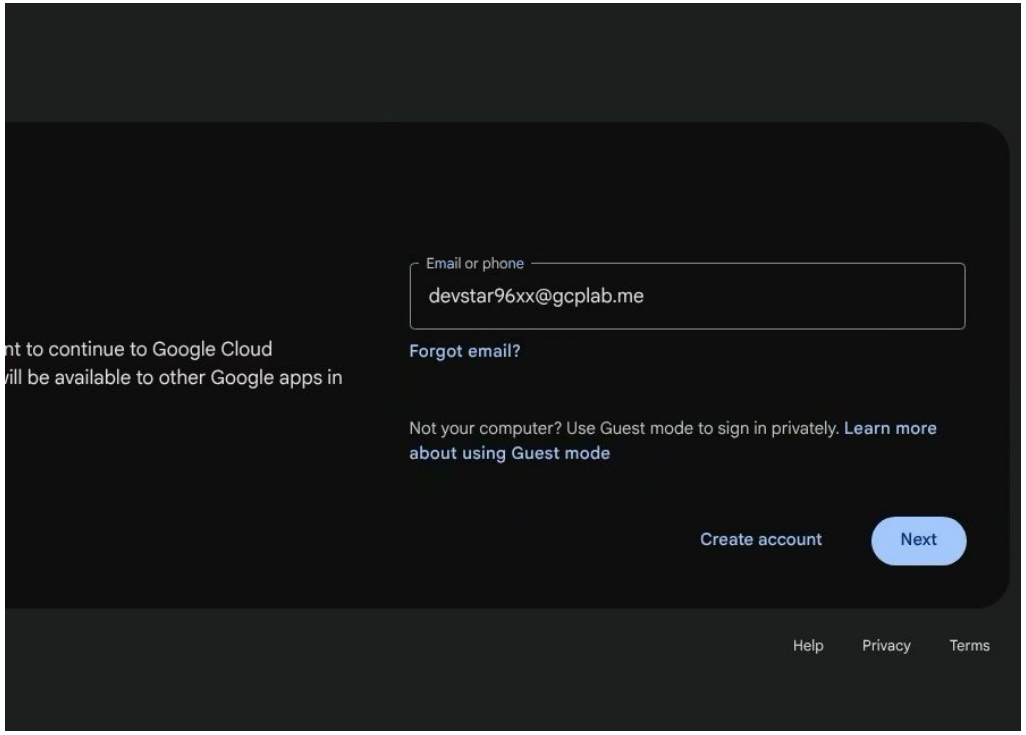
- ✓ Live federation into BigQuery, no pipelines
- ✓ Read-only and write-capable agents, structurally separated
- ✓ Spanner Graph vs. BigQuery Graph — right engine per question

BEFORE YOU START

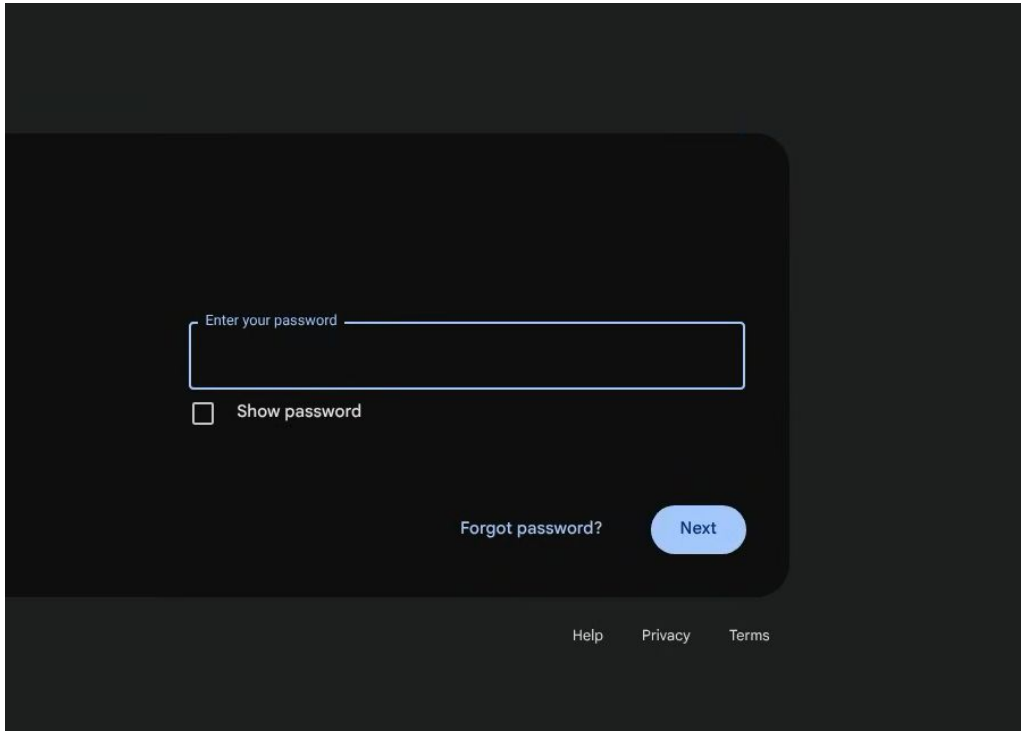
Signing In

Open an Incognito / Private browser window, go to console.cloud.google.com, and sign in with the email and password on your card.

STEP 1 · EMAIL



STEP 2 · PASSWORD



Incognito avoids clashes with anyone's own Google account already signed in on this laptop.

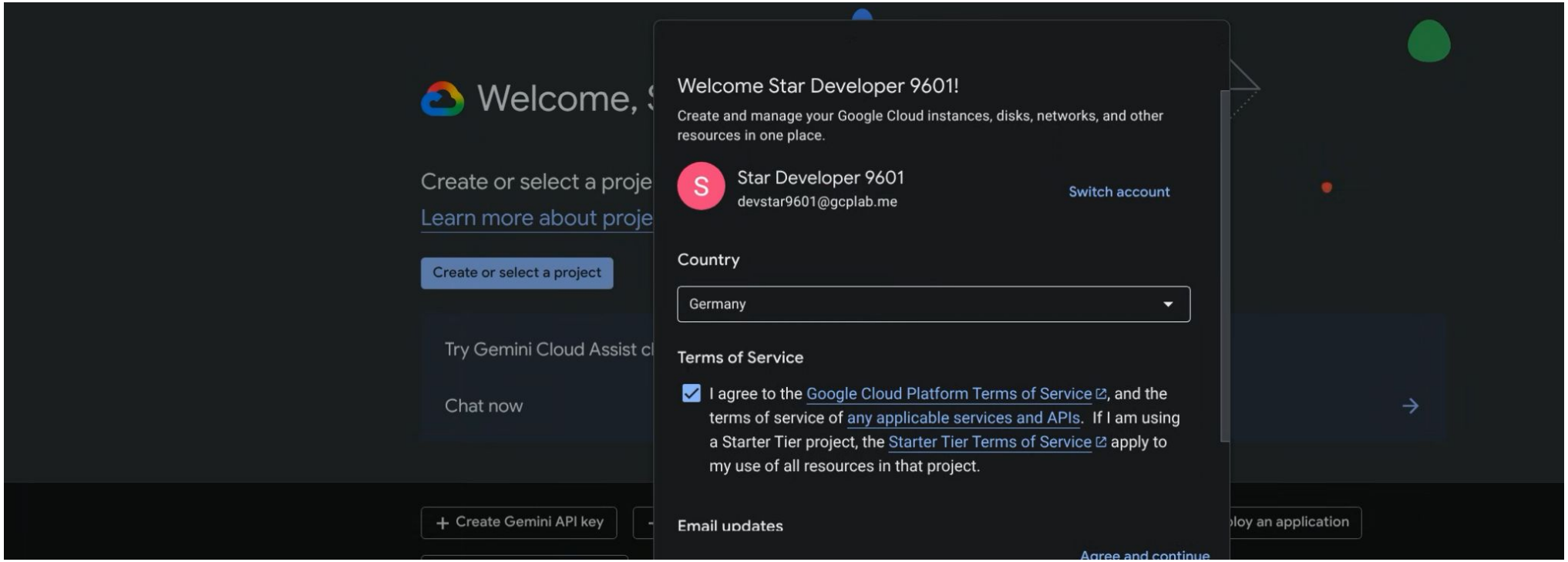
Google Cloud

FIRST LOGIN ONLY

Welcome & Terms of Service

The first time you sign in, the Console greets you by name and asks you to accept the Terms of Service. Check the box and click “Agree and continue.” You'll only see this once.

TERMS OF SERVICE



If the country shown is wrong, it's fine to leave it — it doesn't affect anything in the lab.

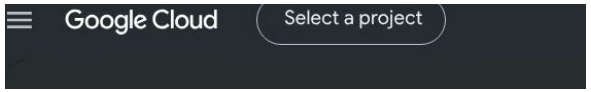
Google Cloud

ALMOST THERE

Selecting Your Project

Click “Select a project” at the top left of the Console, then pick the one project assigned to you — it's named Live Labs Berlin 2026-96XX, matching the number in your email.

1 **STEP 1 · SELECT PROJECT**



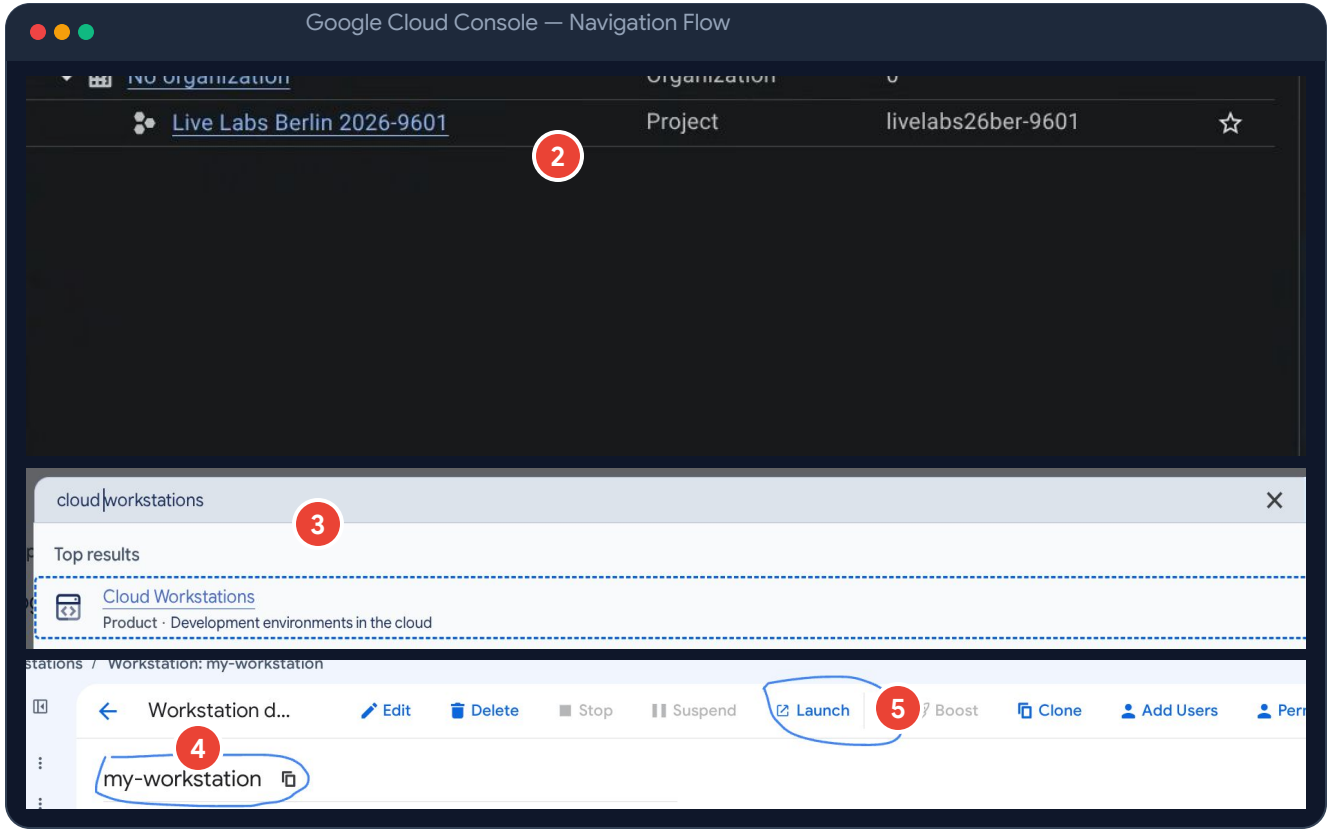
2 **Choose Assigned Project**
Select: **(ai) Live Labs Berlin 2026-96XX**

3 **Search Cloud Workstations**

4 **Choose your Workstation**

5 **Click on Launch**

6 **Open Terminal**
Terminal ⟶ New Terminal



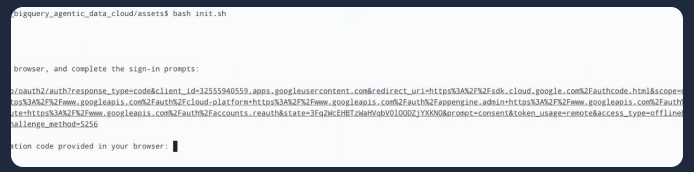
You'll only ever see your own project in this list — each login is scoped to one project.

Google Cloud

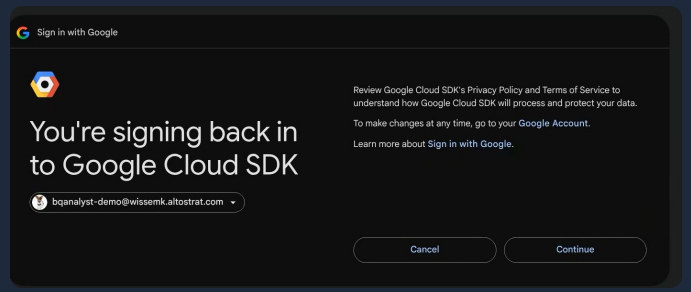
gcloud CLI Authentication Process

Five-step workflow to authorize Google Cloud SDK and complete environment setup

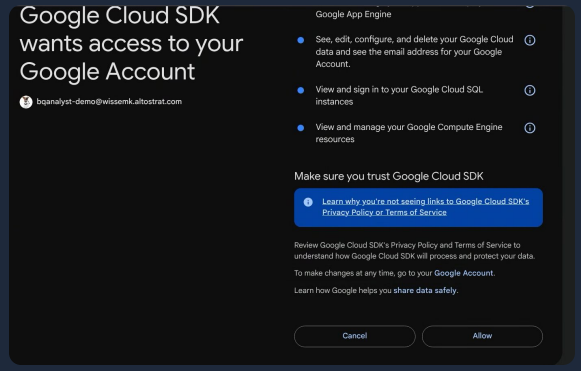
1 Run Prerequisites



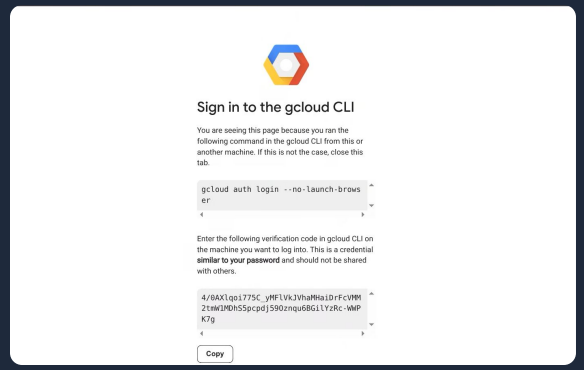
2 Initiate SDK Sign-In



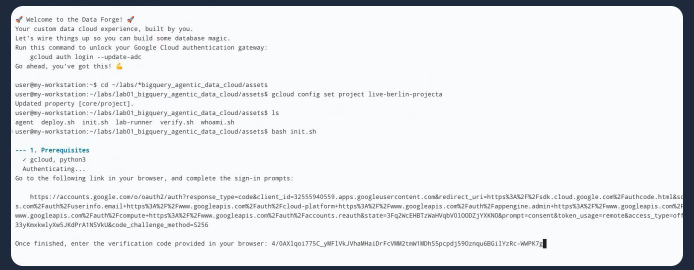
3 Grant Account Access



4 CLI Verification Code



5 Authentication Complete



SIGN IN · AGY

Sign in to the Antigravity CLI (agy) with your Google Cloud project

1. **Start it**

   In the workstation terminal type: agy

2. **Choose the login method**

   Use the arrow keys: “2. Use a Google Cloud project”, then Enter

3. **Sign in in the browser**

   Open the link agy shows. Sign in with your lab account (not your own Google account). In Chrome, “Stay signed out” is fine

4. **Paste the code**

   Allow access, copy the authorization code, paste it in the terminal, Enter

5. **Project and location**

   Project ID: your lab project. **Location: global**

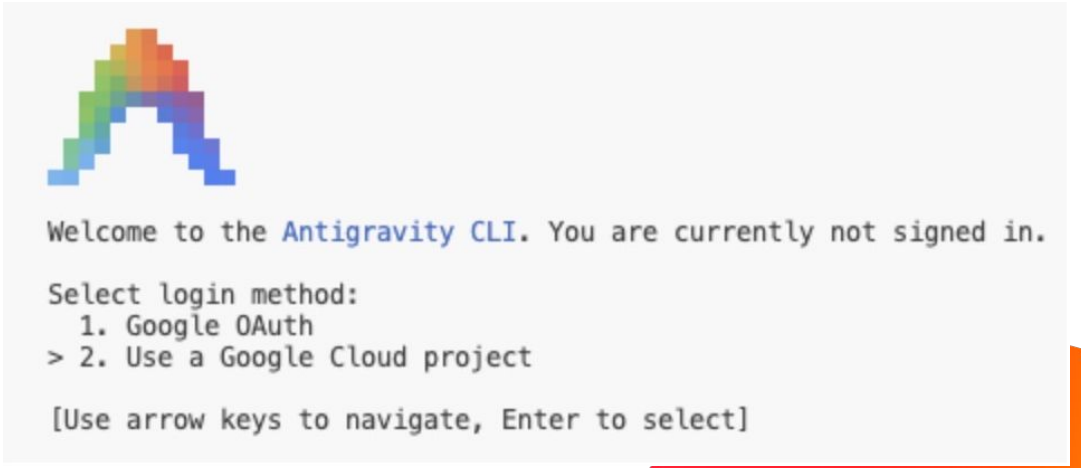
6. **Colour scheme**

   Pick any, leave “Import extensions from Gemini CLI” unticked, then Next

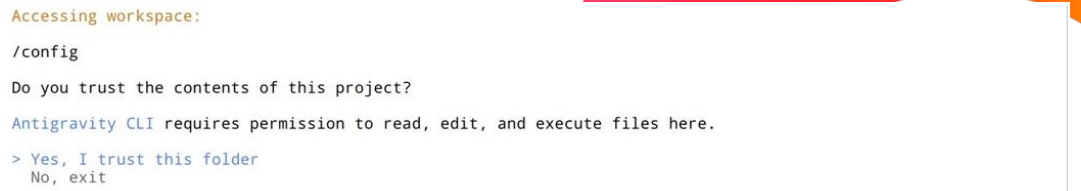
7. **Trust the folder**

   “Do you trust the contents of this project?” → Yes, I trust this folder

You do this once per workstation. The lab guide calls it Step 3; we do it now to save time.



Step 2 — the login choice



Step 7 — trust the folder

Now, still inside agy: /settings → Tool Permission: always-proceed, then /exit.
Later, after bash init.sh (Lab 1, Step 1): agy → /mcp → servers connected.

Google Cloud

YOUR TURN · UNTIL 0:15

Get connected

- [ ] Signed in to the Google Cloud console with your lab account (Incognito window)
- [ ] Cloud Workstations → my-workstation → Start → Launch
- [ ] In the editor: File → Open Folder → `/home/user/labs/lab01_…`
- [ ] Terminal → New Terminal → `cd ~/labs/*bigquery_agentic_data_cloud/assets`
- [ ] Typed `agy` and signed in with your Google Cloud project (location: global)

Wait for the Lab 1 briefing at 0:15. Not working? Raise your hand now — it is the easiest moment to fix it.



UNDER THE HOOD

Four terms you will hear today

01

MCP

Agent ↔ tools & data

How an agent discovers and calls tools — e.g. BigQuery's own MCP server.

Used in both labs

02

ADK

Building the agent

Google's open-source framework for writing agents in code: models, tools, routing.

Used in both labs

03

A2A

Agent ↔ agent

Open protocol for one agent to call another, across frameworks and clouds.

Lab3

04

Data Agent Kit

Agents in your IDE / CLI

Google's extension + plugin that gives coding agents data skills and MCP tools.

GA (Not used in Labs)

UNDER THE HOOD | MCP ARCHITECTURE

MCP — How the Agent Reaches Tools and Data

Open standard (Anthropic, 2024)

Google remote MCP servers

HTTP Protocol & Tool Discovery

A server publishes a list of available tools; the agent's client invokes them over standard HTTP.

Google Managed MCP Endpoints

Hosted services include `bigquery`, `dataplex`, Spanner, AlloyDB, and more.

Tool Filter Guardrail Layer

Lab 1 filters out permissible tools: `execute_sql` (DROP table capability) is never exposed to the model.

IAM Roles & Security Boundary

IAM provides defense-in-depth: the agent's service account holds strictly read-only permissions.

lab 1 · agent/agent.py (condensed)

```
BIGQUERY_READONLY_TOOLS = [
"list_dataset_ids", "get_dataset_info",
"list_table_ids", "get_table_info",
"execute_sql_readonly",
] # execute_sql deliberately absent

def mcp_toolset(url, tool_filter):
return McpToolset(
connection_params=StreamableHTTPConnectionParams(
url=url, timeout=30.0),
header_provider=google_auth_headers, # fresh token
tool_filter=tool_filter, # the guardrail
)

# TODO_2 — what participants write:
BIGQUERY_TOOLSET = mcp_toolset(
BIGQUERY_MCP_URL, BIGQUERY_READONLY_TOOLS)
CATALOG_TOOLSET = mcp_toolset(
CATALOG_MCP_URL, CATALOG_TOOLS)
```

Google Cloud

UNDER THE HOOD · ADK ARCHITECTURE

ADK — The agent is code, and so is the routing

ADK Python 2.0 GA · 19 May 2026

Open Source

Agent Composition

`LlmAgent` = Model + Instruction + Tools list

Classification Router

Router is an `LlmAgent` with `tools=[]` — it classifies user intent without taking direct action.

Deterministic Python Lookup

Specialist selection uses standard dictionary lookups — not AI choice. Routing fixes are reviewed directly in code diffs.

Graph-Based Workflows

ADK 2.0 executes agents as graph nodes, replacing legacy Sequential, Parallel, and Loop structures.

lab 2 · agent/router.py + orchestrator.py

```python
router = LlmAgent(
name="lab1_router",
model="gemini-3.6-flash",
instruction=ROUTER_INSTRUCTION, # TODO_A
tools=[], # can classify, cannot act
)

_INTENT_TO_SPECIALIST = {
"ANALYTICS": bq_analyst_specialist,
"CATALOG": catalog_specialist,
"GRAPH_ANALYTICAL": graph_specialist,
"GRAPH_OPERATIONAL": graph_specialist,
"TRANSACT_ALLOYDB": alloydb_transact_agent, # TODO_B
"TRANSACT_SPANNER": spanner_transact_agent, # TODO_B
}

intent = parse_intent(raw) # one word from router
specialist = _INTENT_TO_SPECIALIST.get(intent)
# no entry -> loud error, never "closest agent"
```

UNDER THE HOOD · A2A

A2A — when one agent calls another

Spec v1.0 · Linux Foundation · Apr 2026

ADK's A2A support

Agent Discovery & Identity

Each agent publishes an Agent Card (name, skills, URL, auth) at `/.well-known/agent-card.json`.

Universal Interoperability

Any framework can call any other: ADK, LangGraph, Microsoft, AWS, Salesforce…

Current Implementation (Lab 2)

Honest note: Lab 2 does NOT use A2A. Its write hop is a plain HTTPS call with a Cloud Run ID token (bottom right).

The A2A Evolution

With A2A, the same hop becomes a `RemoteA2aAgent` pointing at the write service's card.

with A2A · adapted from adk.dev quickstarts

```
# write service: expose the agent over A2A
from google.adk.a2a.utils.agent_to_a2a import to_a2a
a2a_app = to_a2a(root_agent, port=8001)
# card: <url>/.well-known/agent-card.json

# dispatch service: call it like a local sub-agent
from google.adk.agents.remote_a2a_agent import (
    RemoteA2aAgent, AGENT_CARD_WELL_KNOWN_PATH)
spanner_transact = RemoteA2aAgent(
    name="spanner_transact",
    description="Writes orders to Spanner",
    agent_card=f"{TRANSACT_URL}{AGENT_CARD_WELL_KNOWN_PATH}",
)
```

what lab 2 actually does · agent/remote_transact.py

```
token = _id_token(TRANSACT_URL)      # Cloud Run ID token
requests.post(f"{TRANSACT_URL}/transact",
    json={"message": query, "engine": self.engine},
    headers={"Authorization": f"Bearer {token}"})
```

UNDER THE HOOD · DATA AGENT KIT

Data Agent Kit — Data skills inside your coding agent

GA | Not used in the labs

IDE & CLI Extensions

A VS Code extension (pre-installed in Cloud Workstations) plus a plugin for Antigravity CLI, Claude Code, Codex CLI, and Gemini CLI.

Bundled Remote MCP Servers

Bundles skills (markdown guides) with Google's remote MCP servers: BigQuery, Knowledge Catalog, Spanner, AlloyDB, Cloud SQL, Bigtable, Storage, Spark.

Full Transparency & Governance

Asks permission before running steps and explicitly shows each MCP call and the raw SQL it sent.

Standardized Data Workflows

Follows the same pattern as today's labs — skills + MCP — packaged for everyday data engineering & analytics tasks.

install the plugin · from the plugin README

```
# Antigravity CLI
agy plugin install \
https://[link removed]

# Claude Code
claude plugin marketplace add \
https://[link removed]
claude plugin install dak@dak-marketplace

# Codex CLI
codex plugin marketplace add \
GoogleCloudPlatform/data-agent-kit-plugin
codex plugin add dak@dak-marketplace

# then run the setup skill: /dak:dak-setup
# example prompt:
# "Find datasets about customer orders in
# Knowledge Catalog and show their lineage"
```

Google Cloud

UNDER THE HOOD · ANTIGRAVITY

Antigravity - One agent runtime, three ways to drive it

IDE · Antigravity 2.0

Launched 2026

Desktop app for macOS, Windows and Linux. You describe a task; agents plan, run steps in parallel and ask before acting.

```
1. Open the project folder
2. Type the task in agent panel:
"Complete TODO_3 in
agent/agent.py: a catalog-
first instruction that halts
when a term is missing"
3. Review plan & diff, then approve
```

Not on today's workstations (they run Code-OSS + agy)

CLI · agy

Used in both labs

Terminal agent. Pre-installed on workstation; reads AGENTS.md, .agents/skills and MCP config.

```
# install (macOS/Linux)
curl -fsSL https://antigravity.google\
/cli/install.sh | bash

agy # interactive
agy -i "List terms in
ecommerce-glossary"
# ^ what labs use
agy -p "..." # one-shot
```

Lab rule: always agy -i, and approve what it asks to run

SDK · google-antigravity

Python library on the same runtime, for building the agent into your own code and pipelines.

```
pip install google-antigravity

import asyncio
from google.antigravity import Agent
async def main():

async with Agent() as agent:
r = await agent.chat(
"What files in dir?")
print(await r.text())

asyncio.run(main())
```

data-forge uses it in tests/agentic_e2e.py to auto-fix failed deploys

WORKING WITH AGY · 1

Copy the agy prompts exactly — here is why

1 A prompt is a work order

It says exactly what to change, and what not to touch. Change the words and agy may change other code the lab needs.

2 Every “don't” protects a later step

Example: “don't put the revenue formula in the instruction”. If it is there, the agent never needs the catalog, and Step 7 proves nothing.

3 agy can still get it wrong

It is an AI model: the same prompt can give different code. So after each prompt, run the check line from the guide.

Lab 2 · Step 4 · TODO_B prompt

```
agy -i "In agent/orchestrator.py,
add the two missing entries to  INTENT TO SPECIALIST: TRANSACT_ALLOYDB
maps to alloydb transact agent and TRANSACT SPANNER maps to
spanner_transact_agent. Both are already imported.
Do not change any other entry and do not add routing logic to the
router's instruction."
```

What to change → “add the two missing entries … maps to …”

What not to touch → “Do not change any other entry …”

```
Then check:  grep -n -A 10 '^_INTENT_TO_SPECIALIST' agent/orchestrator.py
```

Google Cloud

If the code comes out wrong, how will I know?

1. It stops at once

Easy: the error tells you

A TODO is left empty ⟶ deploy.sh refuses in one second and names the TODO

The Python file is broken ⟶ deploy.sh refuses

2. The demo looks wrong

You notice it in the console

No “stop” rule in TODO_3 ⟶ in Lab 1 Step 7 the agent answers instead of stopping

No commit in TODO_C ⟶ Lab 2 says “success”, but the row count stays the same

3. Nothing looks wrong

Only the check line finds it

A write tool slipped into the tool list ⟶ the lab works, the safety is gone

A made-up status like 'Pending' ⟶ the row is skipped by every revenue query

Revenue restored in your own words ⟶ a different revenue number

That third column is why the guide asks you to check after every prompt.

LAB 1

What You're Building — The Shape of It

The end-to-end flow connecting user questions to intelligent agent decisions and foundational data stores.

SYSTEM ARCHITECTURE

How the components interact

1. Interface Layer

User submits natural language queries via standard browser interface.

2. Decision Engine

The Agent evaluates request intent and determines required execution steps.

3. Foundation Tier

Combines business definitions (Catalog) with analytical data processing (BigQuery).

ENTRY POINT

You — asking a question in your browser

CENTRAL CONTROLLER

The Agent — decides what to do

checks the rulebook, then the data

SEMANTIC RULES

Knowledge Catalog

The Rulebook

Provides business definitions, glossary terms, and governance policies so the agent correctly interprets user intent.

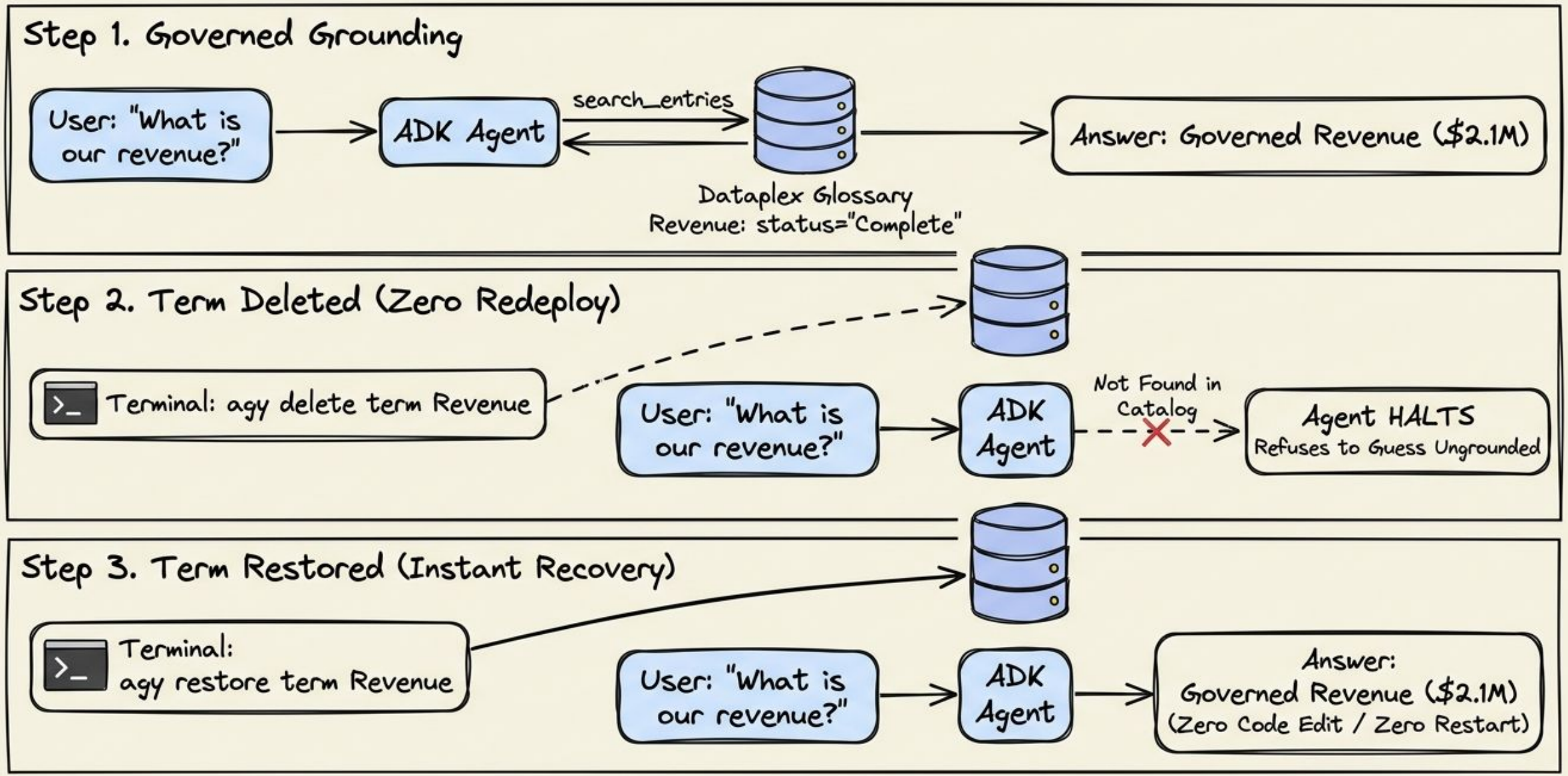
ANALYTICAL ENGINE

BigQuery

The Data

Executes high-speed SQL queries across structured warehouse tables and data lakehouse files to retrieve factual results.

Google Cloud Diagram - Dynamic Data Governance Workflow



HOW THE LAB WORKS · 1

You work in two windows

Terminal BUILD & CHANGE

- Run the setup scripts
- Ask agy to write the code (the TODOs)
- Add, delete or restore glossary terms
- Deploy with bash deploy.sh

Lab console (web page) WATCH & TEST

- Press buttons, one screen per step
- Ask the agent questions
- See what the agent is allowed to see
- Read the results, then go back to the guide

The console does not exist at the start. Your own bash deploy.sh creates it in Step 6.

Simple rule: *Change things in the terminal, check them in the console.*

How the steps depend on each other

Lab 1

init.sh — connects agy → **TODO_2** — the tools → **TODO_3** — the instruction → **deploy.sh** — builds the agent → **Console 6–9** — the demos

Glossary changes (Step 7) need no deploy: the agent reads the catalog every time it is asked.

Lab 2

init.sh — 4 servers → **TODO_A** — the router → **TODO_B** — the dispatch → **TODO_C** — the write → **deploy.sh** — 3 services → **Console 7–13** — the demos

Lab 2 borrows from Lab 1

the tables · the glossary · the email masking · the graph reservation.
So always restore the Revenue term after you delete it — both labs use the same one.

ZOOMING IN · THE DATA

BigQuery

Not just a data warehouse — an agentic, borderless lakehouse. One SQL engine that queries data wherever it lives, in whatever format it's in, and reasons over it with AI.

1. DATA SOURCES & LAKEHOUSE

BigQuery tables
Native storage, today's lab

Cloud Storage files
CSV, JSON, Parquet, ORC

Iceberg tables
Open format, on Google Cloud

Iceberg on AWS & Azure
Cross-Cloud Lakehouse (Preview)

2. AI & ADVANCED ANALYTICS

BigQuery ML
Train & run models in plain SQL

Semantic & vector search
Find rows by meaning

AI functions
AI.GENERATE, AI.FORECAST — call Gemini from SQL

Graph queries
Connected questions — next slide

Why it's in this lab: today you'll use its core — fast SQL over the three ecommerce tables your agent answers from.

ZOOMING IN · THE RULEBOOK

Knowledge Catalog

WHAT IT IS

A place to write down what your data actually means — in plain business language, attached directly to the table or column it describes.

WHY IT'S IN THIS LAB

Your agent reads this definition before it writes any SQL — so it never has to guess what "revenue" means.

STEP 1

Glossary

The top-level container for a business domain.

STEP 2

Categories

Folders that organize terms — e.g. Finance > Revenue.

STEP 3

Terms

Rich definitions attached directly to a table or column.

SEMANTIC SEARCH · POWERED BY GEMINI

A business user searches Knowledge Catalog for **"Active Customer"** and is pointed straight to the technical table that holds it — semantic search, powered by Gemini, understands intent beyond keywords.

LAB NOTE: This is the term you'll delete and restore live, in the halt-condition demo.

ZOOMING IN · WHO'S ASKING

Masking: Same Question, Different Answer

Your agent and the lab runner ask BigQuery the exact same question — but they run as two different identities, and emails only show up for one of them.

```
SELECT name, email FROM customers LIMIT 3;
```

AGENT IDENTITY · MASKED READER
Restricted permissions applied automatically by platform policies

Hannah Lee	xxxxx@example.com
Omar Haddad	xxxxx@example.com
Yuki Sato	xxxxx@example.com

RUNNER IDENTITY · FINE-GRAINED READER
Full unmasked column access granted to authorized runner identity

Hannah Lee	hannah@example.com
Omar Haddad	omar@example.com
Yuki Sato	yuki@example.com

KEY TAKEAWAY: Nothing in the question changed — only who's asking. The clearest proof that rules live in the platform, not the prompt.

ZOOMING IN · CONNECTED QUESTIONS

BigQuery Graph

WHAT IT IS

A way to ask “connected” questions — not just filters, but relationships between rows.

WHY IT'S IN THIS LAB

You'll ask which products tend to sell together — the kind of question a normal SQL query struggles with.

GRAPH STRUCTURE · AT A GLANCE



CORE CONCEPT

A graph is just this: **boxes and the lines between them**, expanded at scale across complex datasets.

KEY INSIGHT: While standard SQL joins tables row-by-row, graph queries traverse relationships natively for multi-hop pattern discovery.

YOUR TURN · 60 MINUTES

Lab 1 — go!

Open user_guide.md in the Lab 1 folder and follow it from Step 1. Aim for these milestones:

Steps 1–5	Terminal	Setup, the problem, the catalog, the two TODOs
Step 6	Terminal → console	deploy.sh, then open the console. The agent answers revenue
Step 7 ★	Both	**Delete Revenue → the agent STOPS. Restore it → it answers again** — KEY MILESTONE
Step 8	Console	Same question, two identities: masked vs real email
Step 9–10	Bonus	Property graph and challenges — if you have time

★ = the moment the lab is built for. Breaks any time. Stuck for 5 minutes? Ask a facilitator. We stop together at 1:35.

PART TWO · LAB 2

Zero-ETL & Orchestration

Once governance is built into the platform, routing agents across three engines becomes a composition problem — not a trust problem.

1. FOUNDATION

Platform Governance

Security and identity rules are enforced directly at the data layer, before agents query.

- Automatic column masking
- Role-based data access
- Zero prompt-level security overhead

2. CORE ARCHITECTURE

Composition Layer

Agents dynamically compose workflows across specialized engines without trust risks.

Key Shift:
Trust Problem → **Composition Problem**

3. SPECIALIZED TARGETS

Three Engines

BigQuery
Analytics & graph pattern discovery

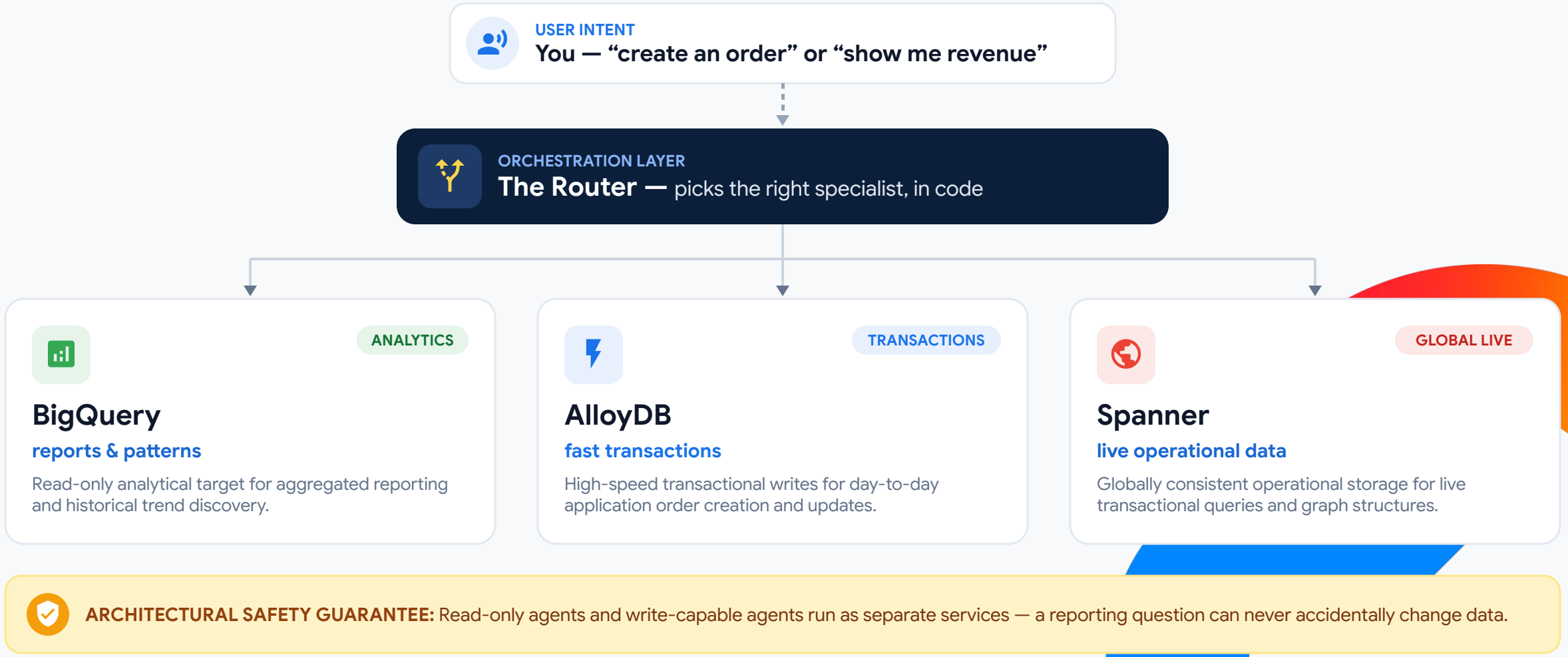
AlloyDB
Fast transactional application workloads

Spanner
Globally consistent live operational data

TAKEAWAY: Routing agents is safe when the platform enforces access control natively across every query target.

PART TWO · LAB 2

What You're Adding — The Shape of It



ARCHITECTURAL SAFETY GUARANTEE: Read-only agents and write-capable agents run as separate services — a reporting question can never accidentally change data.

ZOOMING IN · TWO NEW ENGINES

AlloyDB & Spanner

BigQuery is great at analyzing lots of data at once. These two are built for something different: handling one transaction, instantly.

TRANSACTIONS

1. TRANSACTIONAL ENGINE

AlloyDB

WHAT IT IS

A fast, PostgreSQL-compatible database for day-to-day app transactions.

WHY IT'S HERE IN THIS DEMO

You'll create a new order on it, instantly.

GLOBAL SCALE

2. GLOBAL DISTRIBUTED ENGINE

Spanner

WHAT IT IS

A database that stays consistent even at huge, global scale.

WHY IT'S HERE IN THIS DEMO

You'll create an order on it too, and see it handle connected questions (a graph) built for live data.

ZOOMING IN · NO COPYING

Zero-ETL: Reading Live, Not Copying

Normally, getting data from one system into another means building a pipeline. Here, BigQuery just reaches over and reads the live row — no pipeline, no wait.

AlloyDB

TRANSACTIONS

High-speed transactional source database

Spanner

GLOBAL LIVE

Globally distributed operational database

↓ READ LIVE, INSTANTLY (ZERO-ETL) ↓

BigQuery — sees everything in one view

LIVE FEDERATION

Queries live rows across both AlloyDB and Spanner instantly, eliminating data replication delays.

DEMO STEP: You'll create an order on AlloyDB, then on Spanner, and watch both appear in BigQuery seconds later.

ZOOMING IN · WHO HANDLES WHAT

The Router

One small agent's only job is to read your question and decide which specialist should handle it — a reporting question goes to BigQuery, “create an order” goes to AlloyDB or Spanner.

Routing Examples

DETERMINISTIC

“What's our revenue this month?”

ROUTES TO ➔ Analytics Target — BigQuery specialist

“Create an order for customer 42”

ROUTES TO ➔ Transactional Engine — AlloyDB specialist

Architecture Guarantees

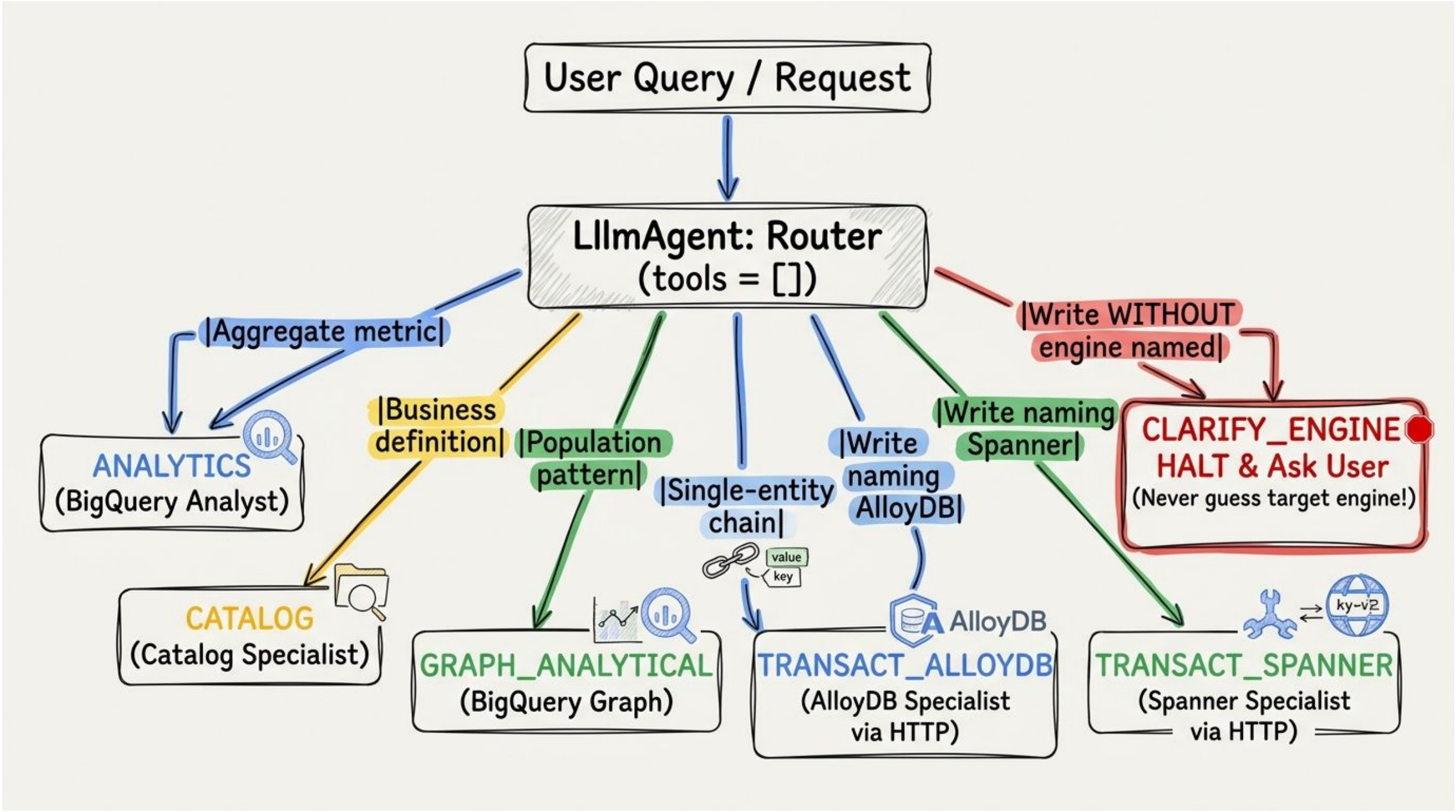
SECURITY & CONTROL

Plain Code Decision

That decision is made in plain code — not left to a model's judgment — so it's always 100% predictable.

Identity & Scoping Checks

Just like in Lab 1, every hand-off passes through identity checks: a specialist only ever sees what you're actually allowed to see.


User Query / Request
LlmAgent: Router
(tools = [])
|Aggregate metric|
|Business definition|
|Population pattern|
|Single-entity chain|
|Write naming AlloyDB|
|Write naming Spanner|
|Write WITHOUT engine named|
ANALYTICS
(BigQuery Analyst)
CATALOG
(Catalog Specialist)
GRAPH_ANALYTICAL
(BigQuery Graph)
value
key
AlloyDB
TRANSACT_ALLOYDB
(AlloyDB Specialist via HTTP)
ky-v2
TRANSACT_SPANNER
(Spanner Specialist via HTTP)
CLARIFY_ENGINE
HALT & Ask User
(Never guess target engine!)

YOUR TURN · 65 MINUTES

Lab 2 — go!

Open the Lab 2 folder (lab02_…), cd assets, and follow user_guide.md from Step 1. Milestones:

Steps 1–5	Terminal	Setup, one query over three databases, the three TODOs
Step 6	Terminal → console	deploy.sh builds three services, then open the Lab 2 console
Step 8 ★	**Console**	**The agent writes an order — and you read it back in BigQuery**
Step 9	Console	The new row is visible at once: no pipeline
Steps 10–13	Bonus	Two graph engines, the safety layers, Lab 1's rules re-checked

• Breaks any time. • Stuck for 5 minutes? **Ask.** • We close together at **15:55**.